This base-page approach is quite common, especially in older versions of .NET, but it still suffers from the problem that you need to remember to change your generated code to ensure the Razor page derives from your custom type rather than `PageModel`. You can always write unit tests that help enforce this (and they would be a good idea in any event).

Ideally, you want to centralize the code that applies authorization to endpoints, so you can tell, at a glance, which parts of the application are protected and to what degree. In chapter 4, you used some extension methods on the `PageConventionCollection` type to add new routes and page route model conventions to the routing system. Additional extension methods exist that enable you to apply authorization to individual pages and whole folders via conventions. Using these, you can establish your authorization rules in one place: the `Program` class. The key methods are

- `AuthorizePage`—Adds authorization to a single page
- `AuthorizeFolder`—Adds authorization to all pages in the specified folder
- `AuthorizeAreaFolder`—Adds authorization to all pages in the specified folder within the specified area

Each of these methods takes the name of the page, folder, and/or area and also includes an overload that takes the name of a *policy*. We will explore policies in detail a bit later. For now, you can think of policies as representing authorization requirements beyond just being authenticated. You can use the `AuthorizePage` method in Program.cs to replace the `Authorize` attribute you applied to the home page earlier.